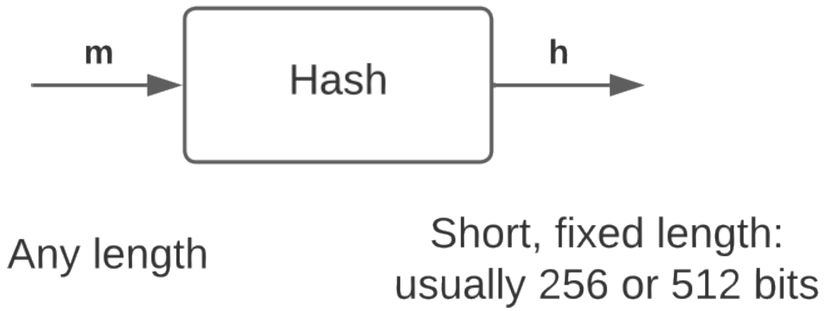
Hashes

Computer Systems Security
2025/26
LEI - UM

Hugo Pacheco hpacheco@di.uminho.pt

Hash Functions

- Hash functions are everywhere
  - Key derivation
  - Digest for authentication
  - Randomness extraction
  - Password protection
  - Commitment schemes
- Not only in crypto:
  - Proofs of work
  - Indexing in version management
  - Deduplication in cloud storage systems
  - File integrity in intrusion detection



One-Way Functions

- Security of cryptographic techniques often translates into establishing that a given function $f : A \rightarrow B$ is **one-way**
  - Given any $a:A$, it is **easy** to compute f(a):B
  - Given any $b:B$ in the range of f, is very **difficult** to find some $x:A$ such that $y=f(x)$ (a pseudo-inverse of y)
- Here, “easy” or “difficult” = existence (or not) of polynomial-time algorithms for their calculation
- It is the core theoretical assumption that makes cryptography with computational security feasible
  - Existence of one-way functions ⇒ P≠NP

One-Way Functions

Cryptographic Hash functions

- A **cryptographic hash function** is a one-way function that maps arbitrary-length bit strings into a fixed-length *n*-bit range

$$H : \{0,1\}^* \rightarrow \{0,1\}^n$$

- Looking at the cardinalities of the domain/codomain
  - No such hash function can be injective (collisions always exist!)
  - But (as one-way functions) finding collisions should be difficult…
- Their use is prevalent in cryptography

Cryptographic Hash functions

Properties

- The requirements of hash functions are usually expressed by the following properties

  1. (**First) pre-image resistant**: given a hash value *h*, it should be unfeasible to obtain a message *m* such that *H(m)=h*

  2. **Second pre-image resistant**: given a message m_1, it should be infeasible to obtain a message m_2 distinct from m_1 such that $H(m_2)=H(m_1)$

  3. **Collision resistant**: it is unfeasible to find distinct messages m_1 and m_2 such that $H(m1)=H(m_2)$

- Increasingly stronger properties ($3 \Rightarrow 2 \Rightarrow 1$), collision resistance subsumes others
  - But some applications may only need one of the weaker properties

One-Way Functions

Birthday paradox

- A standard result in probability theory tells us that we need a reasonably-sized codomain to achieve collision resistance

How many people need to meet to make it more likely than unlikely *that at least 2 share the same birthday?*

- For *H : People* → *Days*, around $\sqrt{366}$ random elements would be enough!
- For typical hash codomain sizes such as 128 and 512 bits
  - It would take around 2^{64} and 2^{256} random picks to find a collision

One-Way Functions

Birthday paradox

- Collisions can be found (in average) with work $\sqrt{2^n} = 2^{\frac{n}{2}}$, much better than brute-force ($2n!$)
    1. Compute domain values like the brute-force attack
    2. Store them in a data structure indexed by image value
    3. Each new image value is searched in data structure
    4. Repeat until a collision is found
- Checking 2^n pairs takes roughly $\sqrt{2^n}$ values
- Overall complexity is that of finding the pre-image of a hash with $\frac{n}{2}$ bits of output (only half of the range)

Cryptographic Hash Functions
Constructions

- Efficient algorithms with nice properties: easy to compute, unpredictable outputs, hard to find pre-images, hard to find collisions
- **Do cryptographic hash functions (really) exist?**
  - They are validated heuristically (similarly to ciphers)
    - International NIST competition to select and evaluate designs
    - Competitors are scrutinized w.r.t. security and performance
    - Several rounds, so more eyes on a small number of proposals

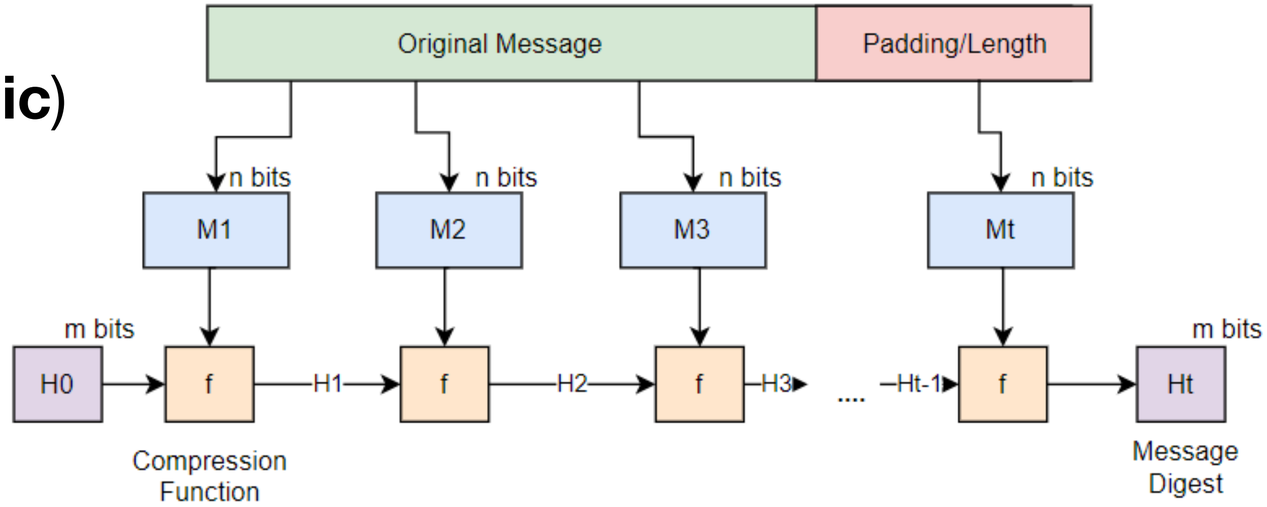
Cryptographic Hash Functions
Constructions

- Two main approaches
  - **Merkle-Damgård construction:** relies on a $m+n$-to-n bits compression function to construct a hash function of output length n for arbitrary input lengths
    - Examples: MD4/5, SHA-n family with $n < 3$
  - **Sponge construction:** uses a l-bit (size of internal state) permutation to construct a hash function for arbitrary input and output lengths
    - Examples: the most recent SHA-3 family

Cryptographic Hash Functions

Merkle-Damgård construction

- All prominent hash functions from 80s-2000s
- *H0* is the initial value (**constant and public**)
- Original message *M* is broken into blocks (*M1*,*M2*,…) of *n* bits
- Compression function *f* applied **iteratively** to each block

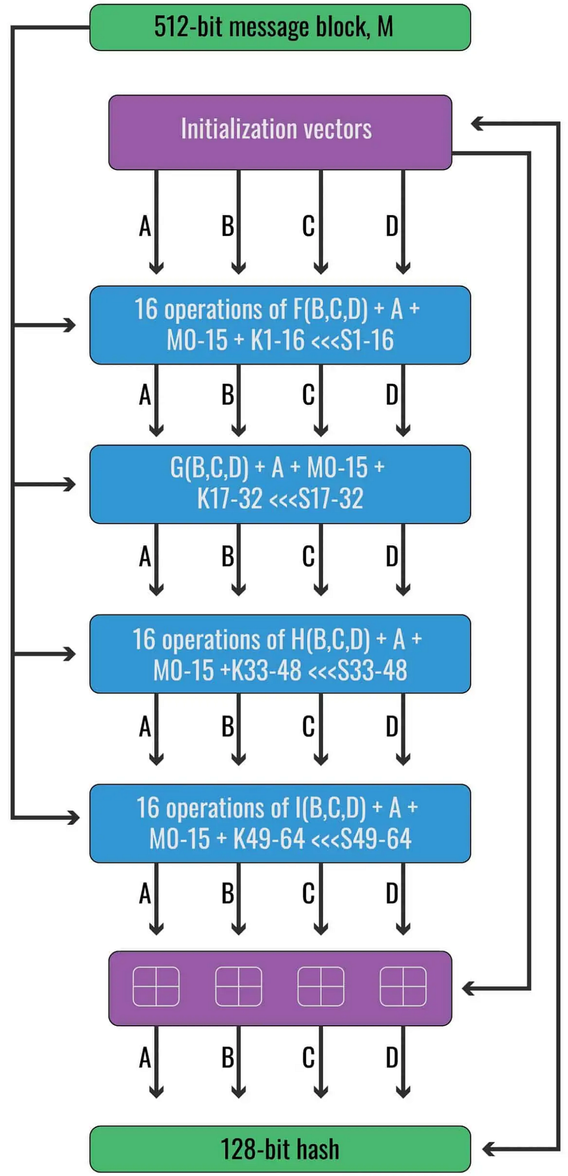


- What if message length is not the same size as the block?
  - Use a padding scheme, typically add *100*… up to *n*

Merkle-Damgård construction

MD5

- Stands for Message Digest (MD)
- **Broken!**
  - These days, it takes seconds to find collisions
- Fixed 128-bit output
- Most popular hash function until broken in 2005
  - Still very popular in non-crypto contexts
    - Due to lower computational requirements



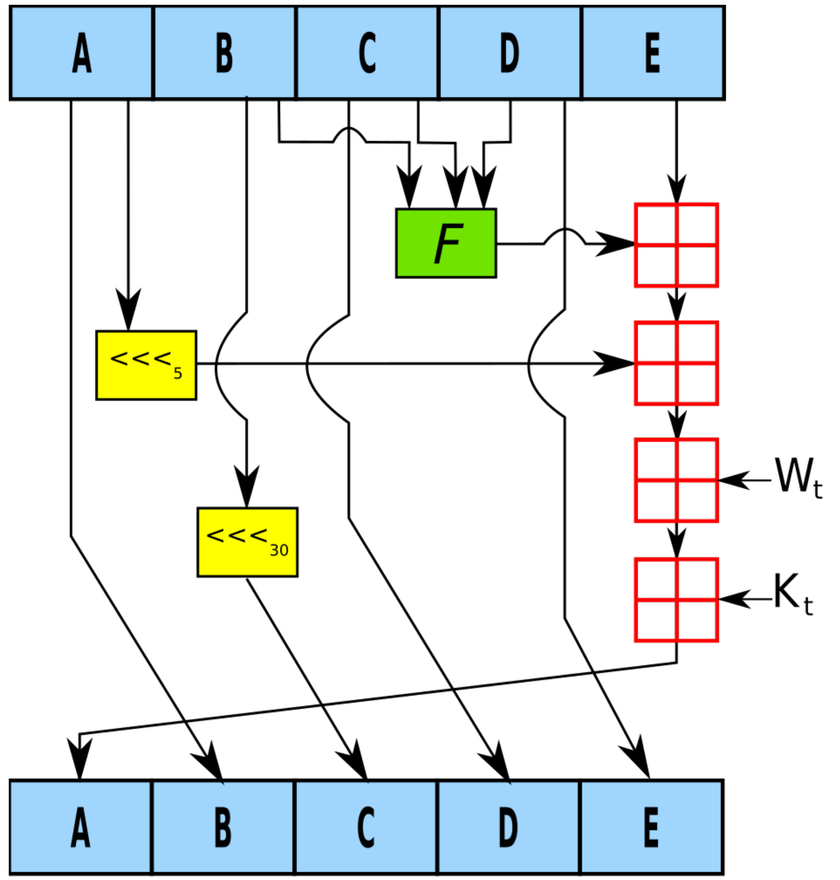
Merkle-Damgård construction
SHA-0

- Stands for Secure Hash Function (SHA)
  - SHA-0 retronym added as new versions were released
  - Standardized by NIST in 1993
- Fixed 160-bit output
- **Broken!**
  - Vulnerability not public at the time it was replaced with SHA-1. Found independently in 1998
    - “A technical flaw which made the algorithm less secure than had been thought has been found in the SHA. NIST is now revising the SHA to correct this flaw.” (FIPS PUB 180-1, 1995)
  - Most recent attacks discover collisions in $2^{33} \ll 2^{80}$ operations

Merkle-Damgård construction

SHA-1

- Published in 1995
  - Small fixes w.r.t. SHA-0 (add rotation $\lll$ to the XOR $\oplus$ for greater diffusion)
- **Broken!**
  - Collisions in $2^{63} \ll 2^{80}$ operations
  - Remained unbroken until quite recently (2017)
- Fixed 160-bit output



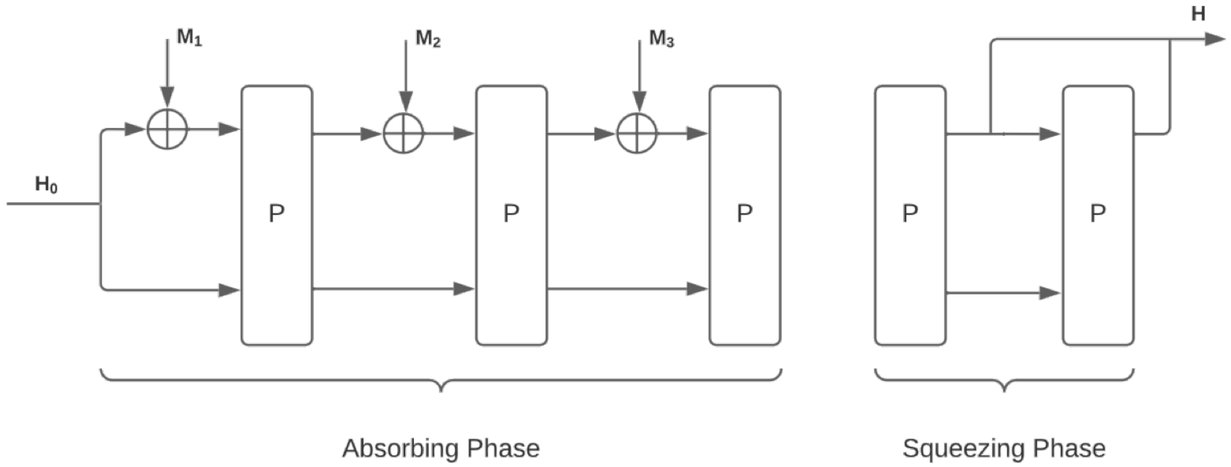
Merkle-Damgård construction
SHA-2

- Published in 2001
- Family of 4 hash functions (SHA-224/256/384/512)
  - SHA-*n*: *n* is the output length
  - SHA-256: block size 512, output size 256 bits
  - SHA-512: block size 1024, output size 512 bits
- Same design principles as SHA-1, larger parameters
- **Still in operation**
  - No known attacks
  - Most applications currently use SHA-2 (256 or 512 bits)

Cryptographic Hash Functions

Sponge construction

- **Absorb** (the input message into the sponge)
  - Fixed initial value *H0*, gradually accumulate message into state
  - Internal state of size $l = r + c$
    - Message broken in blocks of size *r* (rate) that determines throughout
    - Capacity *c* determines the security level (hidden bits of the state)
  - Block $\oplus$'ed into *r* bits of state; only permutation P touches other c bits
- **Squeeze** (output bits from the sponge)
  - Dual process that iteratively constructs output
  - Output constructed block by block



Sponge construction
SHA-3

- Competition called for new design, in case SHA-2 gets attacked
  - Selected in 2009
  - Prudently developed with a different design than SHA-2
  - Drop-in replacements for SHA-2, similar security, SHA-2 still faster
- Sponge applies a 1600-bits permutation
- Family of 4 hash functions (same codomain sizes as SHA-2)
  - SHA3-224: 1600=1152+448
  - SHA3-256: 1600=1088+512
  - SHA3-384: 1600=832+768
  - SHA3-512: 1600=576+1024

Sponge construction
SHAKE

- The main advantage of SHA-3 is that its architecture is much more flexible
- Sponge can be “squeezed” indefinitely to produce hashes with arbitrary output length
- Flexible output size is very useful
  - E.g., to generate keys
  - E.g., as a pseudo-random number generator
- These are known as the SHAKE eXtendable Output Functions (XOFs)
  - SHAKE-128: 1600=1344+256
  - SHAKE-256: 1600=1088+512

Using Hashes

Key-Derivation Functions (KDF)

- Key-Derivation Functions (KDF) allow the derivation of cryptographic keys from other **secret** material
- KDFs are used in a variety of situations
    - Derive keys from weak secrets (passwords/passphrases)
        - Low entropy, not suitable as keys!
    - Derive keys from a “master key”
    - Derive keys of different length from the ones provided
- May accept non-secret inputs, allowing to bind the secret input to application-specific data

Key-Derivation Functions (KDF)

Properties

- KDFs have essentially the same properties as hash functions
  - **Except they are slow to compute on purpose!**
    - E.g., by performing an unusually high number of rounds
- A KDF mitigates the likely weaknesses of its secret inputs
  - By being **resource-intensive** (CPU and/or memory)
    1. Legitimate users spend a moderate amount of time on key derivation
    2. Slows down brute force and dictionary attacks as much as possible

Key-Derivation Functions (KDF)

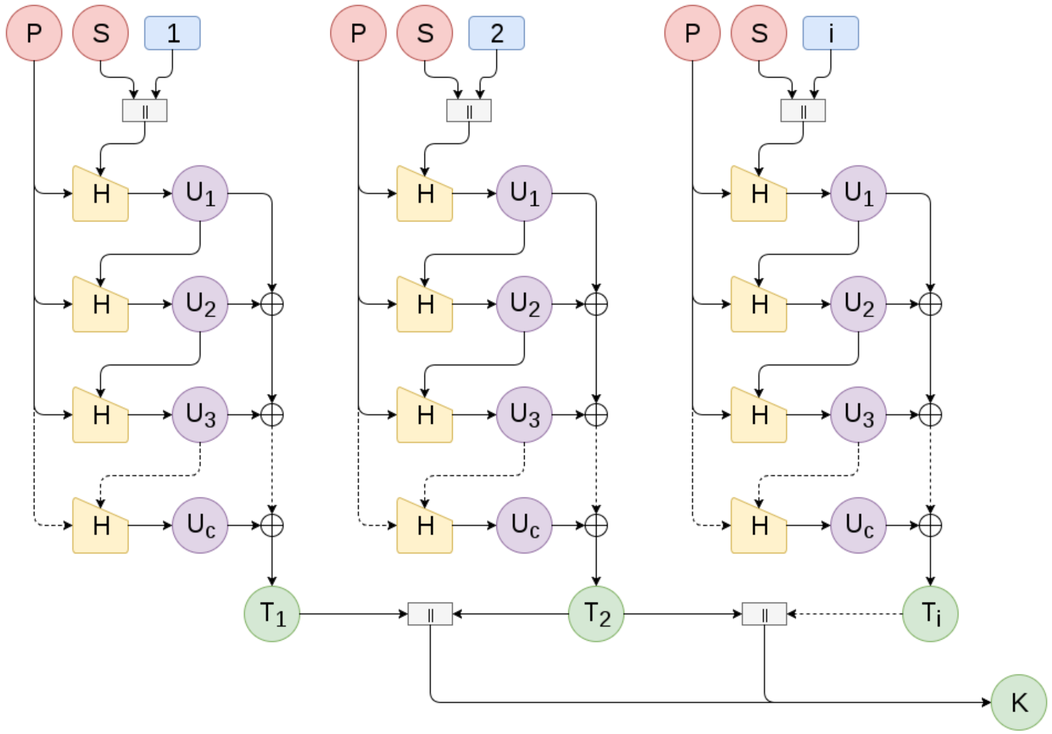
Password-based KDF

- Despite many drawbacks (low entropy, dictionary attacks, brute force attacks)
  - Passwords remain widely used to protect data or control access to resources
  - Cannot be used directly as cryptographic keys
- We can derive cryptographic keys from those passwords using a KDF
  - **But the KDF does not solve all their inherent weaknesses**
    - Attacking the password remains the weakest link
    - An attacker can always run the KDF for obvious passwords

Key-Derivation Functions (KDF)

Password-based KDF

- A password-based KDF shall also
  - Use **salt**, a random numbers that the breaks the determinism of the OW-function
    - To prevent attackers from pre-computing **rainbow tables** of common passwords and their hashed equivalent
- E.g., PBKDF2 produces a key $\mathtt{K}$ from:
  - A user password $\mathtt{P}$
  - A pseudo-random hash function $\mathtt{H}$
  - A salt $\mathtt{S}$ and a iteration count $\mathtt{c}$



Key-Derivation Functions (KDF)

Password-based KDF

- **PBKDF2:** one of the most widely used KDFs in real applications
- **Argon2:** winner of the Password Hashing Competition
- **Bcrypt:** support adjustable cost
- **SCrypt:** extends PBKDF2 to be resistant to hardware brute-force attacks

Algorithm	Year	Memory Usage	GPU Resistance	Side-Channel Resistance	Parallelism
SHA (plain)	1993	Minimal	None	High	Low
PBKDF2	2000	Minimal	Low	High	Medium
Bcrypt	1999	4 KB	Medium	Medium	Low
Scrypt	2009	Configurable (MB)	High	Low	Medium
Argon2i	2015	Configurable (GB)	High	High	High
Argon2d	2015	Configurable (GB)	Very High	Low	High
Argon2id	2015	Configurable (GB)	Very High	Medium	High

Estimated cost of hardware to crack a password in 1 year.

KDF	6 letters	8 letters	8 chars	10 chars	40-char text
DES CRYPT	< $1	< $1	< $1	< $1	< $1
MD5	< $1	< $1	< $1	$1.1k	$1
MD5 CRYPT	< $1	< $1	$130	$1.1M	$1.4k
PBKDF2 (100 ms)	< $1	< $1	$18k	$160M	$200k
bcrypt (95 ms)	< $1	$4	$130k	$1.2B	$1.5M
scrypt (64 ms)	< $1	$150	$4.8M	$43B	$52M
PBKDF2 (5.0 s)	< $1	$29	$920k	$8.3B	$10M
bcrypt (3.0 s)	< $1	$130	$4.3M	$39B	$47M
scrypt (3.8 s)	$900	$610k	$19B	$175T	$210B

On Keys

- Keys are often the most sensitive material a secure system holds
  - Their **shape and size** depend on the specific technique
    - Symmetric crypto: usually random bit strings; sizes ranging between 128 and 256 bits
    - Asymmetric crypto: have a "hidden" mathematical relationship; much larger sizes (RSA takes 4096 bits and ECC takes 400 bits for 128-bit security)
  - Their **quality** is critical
    - Always use a Cryptographic Secure Random-Number Generator (or a KDF if need be)
  - Their **management** (storage, backup, disposal, etc) must be taken with extreme care
    - Ideally, in an external trusted hardware such as a Hardware Security Module (HSM) or a Smartcard

On keys
Handling

- Keys are classified as
  - **Long-term** keys (aka **static** keys)
  - **Short-term** keys (aka **ephemeral** keys)
- **As a rule of thumb**
  - The key lifetime shall be as short as possible
  - And inversely proportional to their use and the criticality of the protected data
- As we will see, symmetric cryptography favours short-term keys
  - E.g. the establishment of a secure-channel should rely on session keys
- At the programming level, cryptographic APIs typically handle keys as an *abstract datatype*
  - Allows strict control on the available operations to manipulate keys
  - Supports specific safeguards (e.g. clear the memory after use)

On keys
Guidelines

- **Keys shall not be used for multiple purposes**
  - Use instead KDFs to derive different keys
  - Ideally from a master key stored in trusted hardware
- **Keys should be deleted when compromised or expired**
  - But different applications can adopt different policies
- **A *threshold secret-sharing scheme* should be used for key backup**
  - Such that no single entity can recover the key

Using Hashes

Git

```
commit 58fa56e4124b2eb362e61fe0268f660f2c01b8bc
Author: hpacheco <hpacheco@di.uminho.pt>
Date: Fri Feb 17 10:14:40 2026 -0800
Sample Commit
```

- Git stores SHA-1 file hashes
  - Same use as a regular hash, to allow checking file correctness
- Git stores SHA-1 commit hashes
  - To uniquely identify commits
- Why does it use a **cryptographic** hash function?
  - Files: to promote deduplication (file = content), collisions would lead to space duplication
  - Commits: to safely merge histories, collisions would lead to data loss / inconsistent state

Using Hashes

Merkle trees

- Another popular use of hashes is to construct data structures known as **Merkle trees**
  - The founding concept behind **blockchains**
  - Main property is **immutability** (past data is unchangeable)
- E.g., Git histories form a Merkle tree
  - Each commit ID is a hash of
    - Commit message
    - Hash of the parent commit
    - Hash of the directory tree